Введение

Практикум курса состоит из восьми лабораторных работ на собственном стенде из двух виртуальных машин. Работы образуют единую цепочку: приложение проходит путь от запуска на голой виртуальной машине до развёртывания в кластере Kubernetes с доставкой из Git и наблюдаемостью. Каждая работа опирается на результат предыдущей, поэтому порядок выполнения обязателен.

Практикум выдаётся двумя документами:

- **методические указания** — этот документ. Общий для потока справочник на весь семестр: порядок работ и оценивание, требования к стенду и порядок его развёртывания, развёртывание Git-платформы, модель угроз, инструкция ИИ-агенту, литература. Заданий он не содержит;
- **задание** — отдельный документ со своим вариантом: параметры варианта и восемь работ. Задание ссылается на разделы методических указаний по названию.

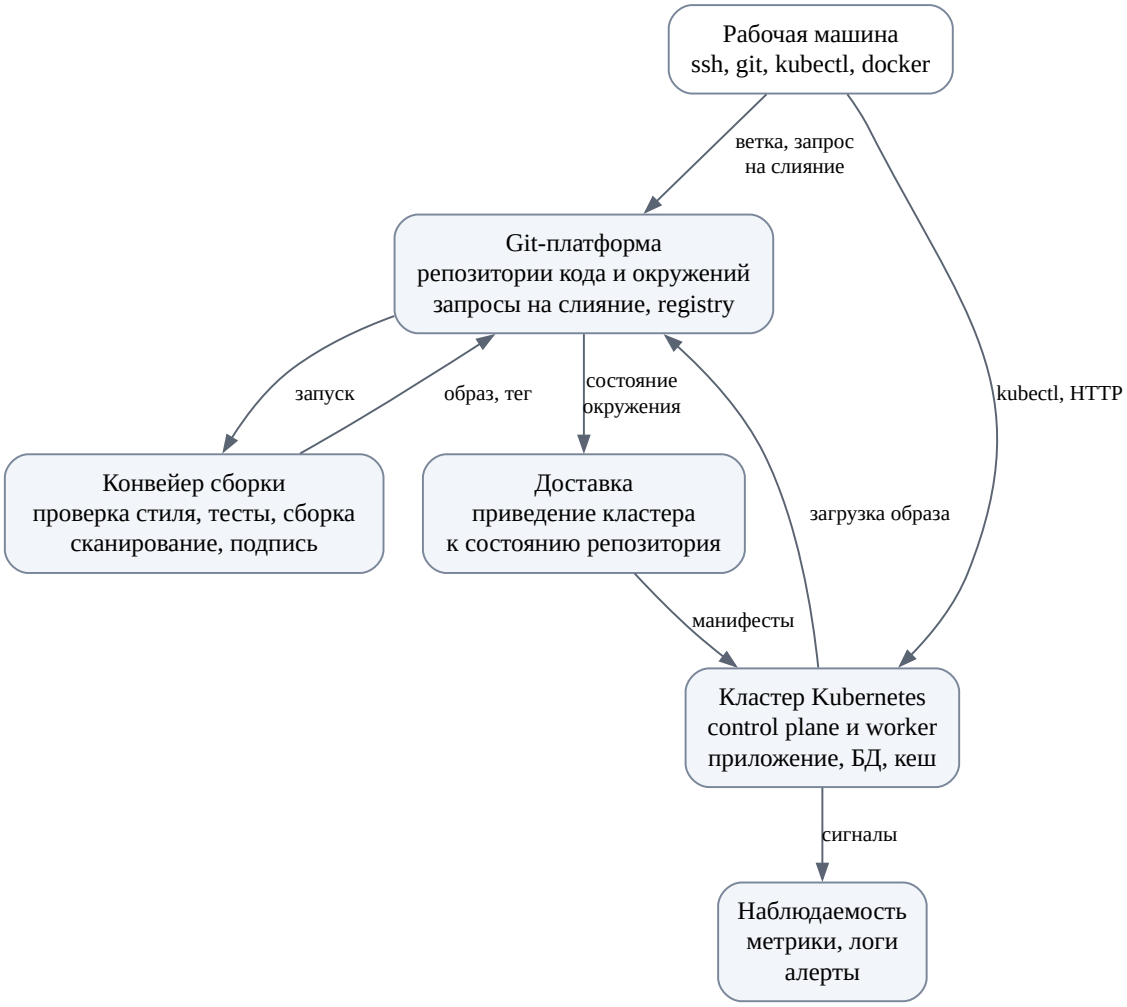


Рисунок 1.1: Схема взаимодействия компонентов.

Набор инструментов задан вариантом и не меняется в течение семестра.

Баллы и сроки

Шкала стобалльная. Основные задания дают 80 баллов, дополнительные — до 12, итоговый тест — 20; сумма выше 100 усекается до 100. Баллы не округляются.

Контрольная точка	Основное	Дополнительное	Неделя
ЛР1 Стенд курса и модель угроз	4	—	1
ЛР2 Git-платформа и процесс	8	1	2
ЛР3 Контейнеризация приложения	10	2	3
ЛР4 CI и DevSecOps	12	2	4
ЛР5 Инфраструктура как код	12	2	5
ЛР6 Оркестрация	12	2	6
ЛР7 CD и GitOps	12	2	7
ЛР8 Наблюдаемость и надёжность	10	1	8
Итоговый тест	20	—	9

Таблица 1.1: Контрольные точки и их вес.

Коэффициент за срок сдачи:

- работы 1–7: сдача на занятии своей недели или следующей — 100 %, на любом занятии до девятого включительно — 80 %, на зачётной сессии — 60 %;
- работа 8: занятия 8 и 9 — 100 %, зачётная сессия — 80 %;
- дополнительное задание принимается вместе с основным и с тем же коэффициентом.

Условие зачёта: не менее 60 баллов, приняты основные задания работ 1–7 и написан итоговый тест.

Литература

- Ким Дж., Хамбл Дж., Дебуа П., Уиллис Дж. Руководство по DevOps. — Манн, Иванов и Фербер, 2018.
- Форсгрен Н., Хамбл Дж., Ким Дж. Ускоряйся! Наука DevOps. — Альпина ПРО, 2023.
- Бейер Б. и др. Site Reliability Engineering. — Google, 2016; полный текст — sre.google/books.
- Райс Л. Безопасность контейнеров. — Питер, 2021.
- Мейер Б., Хохштейн Л., Мозер Р. Запускаем Ansible. 3-е изд. — ДМК Пресс, 2023.
- Пивотто Ж., Бразил Б. Запускаем Prometheus. — ДМК Пресс, 2023.
- Баланов А. Н. DevOps: интеграция и автоматизация. — Лань, 2024; ЭБС «Лань» по подписке университета.
- Шостак А. Threat Modeling: Designing for Security. — Wiley, 2014.

Документация инструментов: Kubernetes, OpenTofu, Ansible, Helm, Argo CD, Flux, OpenTelemetry, cloud-init.

Разработка от спецификации: OpenSpec, spec-kit.

Свойства приложения и безопасность поставки: двенадцать факторов, OWASP Top 10 CI/CD Security Risks, SLSA.

Стенд курса

Стенд состоит из двух виртуальных машин и создаётся один раз в ЛР1. Все работы обращаются к узлам по именам `node-1` и `node-2`, поэтому способ создания машин на задания не влияет.

Требования к стенду

Узел	vCPU	Память	Диск	Роль
`node-1`	4	6 ГБ (8 ГБ при варианте GitLab)	40 ГБ	Git-платформа, registry, runner; с ЛР6 — управляющий узел кластера
`node-2`	2	4 ГБ	20 ГБ	worker кластера

Таблица 2.1: Узлы стенда; размеры — минимальные.

Операционная система обоих узлов — Debian 13 (trixie), архитектура amd64. Управляющий узел кластера принимает рабочую нагрузку наравне с `node-2`: в ЛР8 узел `node-2` выключается, и приложение вместе со стеком наблюдаемости продолжает работать на `node-1`.

Узлы находятся в одной приватной сети с фиксированными адресами; трафик между ними не ограничивается. С рабочей машины доступны только перечисленные порты.

Порт	Узел	Назначение	Работа
22	оба	sshd узла	ЛР1
80, 443	оба	до ЛР6 — приложение из Compose на `node-1`, с ЛР6 — входной шлюз кластера на обоих узлах	ЛР3
2222	`node-1`	SSH Git-платформы, развёрнутой в контейнере; при установке платформы пакетом на хост Git-доступ идёт через sshd узла на порту 22	ЛР2
8443	`node-1`	Git-платформа и её registry по HTTPS	ЛР2
5050	`node-1`	registry GitLab, только для варианта GitLab	ЛР2
6443	`node-1`	API кластера	ЛР6

Таблица 2.2: Порты, открытые с рабочей машины.

Рабочая машина — ноутбук или настольный компьютер. Движок Docker на ней не требуется: образы собираются на `node-1` через контекст `docker context`. Всё, чем работает студент, ставится на рабочую машину — к той работе, в которой впервые нужно. На узлы ничего ставить руками не нужно: Docker приходит с файлом первичной настройки, остальные службы разворачиваются заданиями работ.

Работа	Что установить на рабочей машине
ЛР1	`ssh`, `ssh-keygen`, `git`, `curl`, `nc`, `openssl`, `cloud-image-utils` (команда `cloud-localds`)
ЛР2	`gitleaks`
ЛР3	`docker` — клиент с `compose` и `buildx`; сканер варианта: `trivy` либо `grype` вместе с `syft`
ЛР4	`cosign`
ЛР5	`ansible`, `ansible-lint`, `ansible-vault`, `tofu`, `yq`
ЛР6	`kubectl`
ЛР7	`helm`, `kubeseal`

Таблица 2.3: Инструменты рабочей машины по работам.

Архитектура стенда — amd64 на любой рабочей машине. На Apple Silicon виртуальные машины amd64 запускаются через эмуляцию. Если ресурсов рабочей машины не хватает, вариант заменяется по согласованию с преподавателем на первом занятии.

Развёртывание стенда

Подходит любой гипервизор с поддержкой cloud-init: VirtualBox, VMware Workstation или Fusion, Hyper-V, libvirt/QEMU, UTM.

1. Скачайте официальный облачный образ Debian 13 (`debian-13-genericcloud-amd64`): он рассчитан на cloud-init и не требует установки системы.
2. Создайте две виртуальные машины с параметрами из таблицы требований, каждой — свой диск из образа.
3. Заведите приватную сеть с диапазоном `192.168.56.0/24` (в VirtualBox — host-only, в libvirt и Hyper-V — своя сеть с этим диапазоном) и закрепите адреса: `node-1` — `192.168.56.11`, `node-2` — `192.168.56.12`. Диапазон зафиксирован: адреса входят в правила firewall и в сертификат Git-платформы.
4. Соберите для каждой машины ISO с настройками: `user-data` из раздела «Файл первичной настройки» и `meta-data` с именем узла. Подключите ISO как второй привод.

```
# cloud-localds входит в пакет cloud-image-utils
printf 'instance-id: node-1\nlocal-hostname: node-1\n' > meta-data
cloud-localds -v seed-node-1.iso user-data-node-1 meta-data
```

На Windows команды выполняются в WSL2, готовый ISO подключается к машине средствами гипервизора. На macOS `cloud-localds` нет — образ собирается штатной утилитой из каталога с двумя файлами:

```
mkdir cidata && cp user-data-node-1 cidata/user-data && cp meta-data cidata/
hdiutil makehybrid -iso -joliet -default-volume-name cidata \
  -o seed-node-1.iso cidata
```

5. Запустите машины. После первой загрузки узлы настроены: пользователи, ключи, firewall и Docker установлены файлом настройки, ручные шаги не требуются.
6. Пропишите имена узлов на рабочей машине (`/etc/hosts`, в Windows — `%SystemRoot%\System32\drivers\etc\hosts`) и на обоих узлах:

```
192.168.56.11 node-1 dev.node-1 prod.node-1
192.168.56.12 node-2 dev.node-2 prod.node-2
```

Имена окружений понадобятся с ЛР6: по ним входной шлюз кластера отличает `dev` от `prod`, а два адреса позволяют обратиться к приложению через любой узел.

7. Выполните проверку из раздела «Проверка стенда» и сделайте снимок состояния обеих машин.

Сеть с NAT также допустима при условии проброса перечисленных портов на рабочую машину.

Файл первичной настройки

Это файл `user-data` формата cloud-init: гипервизор отдаёт его машине при первой загрузке, и служба cloud-init приводит систему в описанное состояние. Файл одинаков для обоих узлов; различаются имя узла, адрес соседа в правилах firewall и порты: на `node-2` порты 2222, 8443, 5050 и 6443 не открываются. Подставьте имя узла, открытый ключ и хеш пароля (`openssl passwd -6`).

Первая часть — имя узла и учётные записи. Пользователь `student` входит только по ключу, а `sudo` запрашивает пароль: на узле, где пользователь не в группе `docker`, с `NOPASSWD:ALL` украденный SSH-ключ дал бы права root. На стенде `student` входит в группу `docker` (третья часть файла), а участник этой группы равен root: это допущение стенда ради работы с `docker` без `sudo` и через `docker context`, поэтому ключ `student` охраняется как доступ root. Пароль задаётся хешем — открытый пароль в файле настройки попадает в историю образа и в репозиторий. Пользователь `app` принадлежит приложению: без `sudo`, без пароля и без входа по SSH.

```yaml
#cloud-config
hostname: node-1
fqdn: node-1
preserve_hostname: false

# Вход только по ключу.
ssh_pwauth: false

users:
  - name: student
    groups: [sudo]
    shell: /bin/bash
    lock_passwd: false
    passwd: "$6$ЗАМЕНИТЕ_НА_ХЕШ"
    sudo: "ALL=(ALL) ALL"
    ssh_authorized_keys:
      - "ssh-ed25519 AAAA... student@laptop"
  - name: app
    shell: /usr/sbin/nologin
    lock_passwd: true
```

Вторая часть — пакеты и файлы. `write_files` кладёт содержимое на диск до первого запуска служб: ограничение размера журналов Docker и репозиторий, из которого он ставится.

```
package_update: true
packages:
  - ufw
  - ca-certificates
  - curl
  - git

write_files:
  - path: /etc/docker/daemon.json
    content: |
      {
        "log-driver": "json-file",
        "log-opts": {"max-size": "10m", "max-file": "3"}
      }
   - path: /etc/apt/sources.list.d/docker.sources
    content: |
      Types: deb
      URIs: https://download.docker.com/linux/debian
      Suites: trixie
      Components: stable
      Signed-By: /etc/apt/keyrings/docker.asc
```

Третья часть — команды первой загрузки: Docker ставится из своего репозитория, затем включается firewall. Соседний узел открыт целиком — трафик между узлами не ограничивается; рабочей машине доступны только перечисленные порты.

```
runcmd:
  - [install, -m, "0755", -d, /etc/apt/keyrings]
  - curl -fsSL https://download.docker.com/linux/debian/gpg
      -o /etc/apt/keyrings/docker.asc
  - chmod a+r /etc/apt/keyrings/docker.asc
  - apt-get update
  - apt-get install -y docker-ce docker-ce-cli containerd.io
      docker-buildx-plugin docker-compose-plugin
  - usermod -aG docker student
  - ufw default deny incoming
  - ufw default allow outgoing
  # Адрес соседа: на node-2 здесь 192.168.56.11.
  - ufw allow from 192.168.56.12
  - ufw allow 22/tcp
  - ufw allow 80,443/tcp
  # Строка только для node-1.
  - ufw allow 2222,8443,5050,6443/tcp
  - ufw --force enable
```

Проверка стенда

Проверка выполняется с рабочей машины и повторяется при сдаче ЛР1.

```
ssh student@node-1 'hostname; id student; id app'
ssh student@node-2 'hostname; id student; id app'
ssh -o PreferredAuthentications=password student@node-1 true   # отказ
ssh student@node-1 'docker run --rm hello-world'               # без sudo
ssh student@node-2 'docker run --rm hello-world'               # без sudo
ssh student@node-1 'ping -c1 node-2'                           # связность узлов
nc -z -w2 node-2 8443                                          # отказ: порт node-1
nc -z -w2 node-1 8443                                          # после ЛР2
```

Результат: вход по ключу работает на обоих узлах, парольный вход отклоняется, `docker` работает без `sudo` на обоих узлах, узлы видят друг друга по именам. Порт, которого нет в таблице для этого узла, с рабочей машины закрыт: проверка `node-2` на 8443 обязана завершиться отказом.

Снимок состояния

После проверки сделайте снимок состояния обеих машин и укажите дату. Снимок — точка возврата: работу можно переиграть, не пересоздавая стенд. Снимок обновляется после каждой сданной работы.

Git-платформа

Раздел общий для потока: платформа задана вариантом, а порядок развёртывания одинаков. Файлы платформы размещаются в каталоге `~/platform` на `node-1`. Способ развёртывания произволен; ниже приведён рабочий пример на Compose, рассчитанный на перечень портов стенда и собственную CA.

Свой CA и сертификат узла

Платформа и её registry работают по HTTPS с сертификатом, подписанным собственным CA. Ключ CA — секрет: он остаётся на `node-1` и не попадает в Git.

```
# Корневой сертификат курса.
openssl req -x509 -newkey rsa:4096 -sha256 -days 400 -nodes \
  -keyout ca.key -out ca.crt -subj "/CN=DevOps course CA"

# Ключ и запрос для node-1: имя узла и адрес указываются в SAN.
openssl req -newkey rsa:2048 -nodes -keyout node-1.key -out node-1.csr \
  -subj "/CN=node-1" \
  -addext "subjectAltName=DNS:node-1,IP:192.168.56.11"

# Подпись запроса собственной CA с переносом SAN в сертификат.
openssl x509 -req -in node-1.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -sha256 -days 365 -copy_extensions copyall -out node-1.crt
```

Доверие к CA:

```
# на обоих узлах
sudo cp ca.crt /usr/local/share/ca-certificates/devops-ca.crt
sudo update-ca-certificates
sudo systemctl restart docker        # демон читает системные CA при старте
```

На рабочей машине CA добавляется в системное хранилище: в Linux — тем же способом, в Windows — в «Доверенные корневые центры сертификации», в macOS — в «Связку ключей». В ЛР5 установка CA на узлы описывается ролью Ansible, в ЛР6 — для среды выполнения контейнеров кластера.

> **Предупреждение.** Отключение проверки сертификата (`insecure-registries`, `-k`, `GIT_SSL_NO_VERIFY`) не допускается: при сдаче проверяется доверие к CA.

Развёртывание Forgejo

Данные хранятся в SQLite, отдельная СУБД не требуется. Встроенный registry пакетов доступен по адресу веб-интерфейса.

```yaml
services:
  forgejo:
    image: codeberg.org/forgejo/forgejo:16.0.3
    container_name: forgejo
    restart: unless-stopped
    environment:
      FORGEJO__server__PROTOCOL: https
      FORGEJO__server__DOMAIN: node-1
      FORGEJO__server__ROOT_URL: https://node-1:8443/
      FORGEJO__server__HTTP_PORT: "3000"
      FORGEJO__server__CERT_FILE: /certs/node-1.crt
      FORGEJO__server__KEY_FILE: /certs/node-1.key
      FORGEJO__server__START_SSH_SERVER: "true"
      FORGEJO__server__SSH_LISTEN_PORT: "22"
      FORGEJO__server__SSH_DOMAIN: node-1
      FORGEJO__server__SSH_PORT: "2222"
      FORGEJO__service__DISABLE_REGISTRATION: "true"
      FORGEJO__packages__ENABLED: "true"
    volumes:
      - ./data:/data
      - ./certs:/certs:ro
    ports:
      - "8443:3000"
      - "2222:22"
```

Развёртывание Gitea

Устройство то же: SQLite внутри контейнера, registry пакетов на адресе веб-интерфейса.

```yaml
services:
  gitea:
    image: gitea/gitea:1.27.3
    container_name: gitea
    restart: unless-stopped
    environment:
      GITEA__server__PROTOCOL: https
      GITEA__server__DOMAIN: node-1
      GITEA__server__ROOT_URL: https://node-1:8443/
      GITEA__server__HTTP_PORT: "3000"
      GITEA__server__CERT_FILE: /certs/node-1.crt
      GITEA__server__KEY_FILE: /certs/node-1.key
      GITEA__server__START_SSH_SERVER: "true"
      GITEA__server__SSH_LISTEN_PORT: "22"
      GITEA__server__SSH_DOMAIN: node-1
      GITEA__server__SSH_PORT: "2222"
      GITEA__service__DISABLE_REGISTRATION: "true"
      GITEA__packages__ENABLED: "true"
    volumes:
      - ./data:/data
      - ./certs:/certs:ro
    ports:
      - "8443:3000"
      - "2222:22"
```

Развёртывание GitLab CE

GitLab CE требует 8 ГБ памяти на `node-1`. Registry — отдельный сервис на порту 5050: сертификат один, слушателей два.

```
services:
  gitlab:
    image: gitlab/gitlab-ce:19.3.1-ce.0
    container_name: gitlab
    restart: unless-stopped
    hostname: node-1
    shm_size: 256m
    environment:
      GITLAB_OMNIBUS_CONFIG: |
        external_url 'https://node-1:8443'
        registry_external_url 'https://node-1:5050'
        nginx['ssl_certificate'] = '/etc/gitlab/ssl/node-1.crt'
        nginx['ssl_certificate_key'] = '/etc/gitlab/ssl/node-1.key'
        registry_nginx['ssl_certificate'] = '/etc/gitlab/ssl/node-1.crt'
        registry_nginx['ssl_certificate_key'] = '/etc/gitlab/ssl/node-1.key'
        gitlab_rails['gitlab_shell_ssh_port'] = 2222
        prometheus_monitoring['enable'] = false
        puma['worker_processes'] = 2
     volumes:
      - ./config:/etc/gitlab
      - ./logs:/var/log/gitlab
      - ./data:/var/opt/gitlab
      - ./certs:/etc/gitlab/ssl:ro
    ports:
      - "8443:8443"
      - "5050:5050"
      - "2222:22"
```

Первый запуск занимает несколько минут; готовность контролируется по `docker compose logs -f`.

Первый вход и registry

Версии в примерах — релизы, актуальные на момент издания. Допускается более свежая версия, но тоже закреплённая: `latest` не используется нигде в курсе, и платформа не исключение. При сдаче потребуется назвать установленную версию. Различие закреплённого и плавающего тега разбирается в ЛР3.

1. Откройте `https://node-1:8443` и создайте администратора; регистрация остальных пользователей выключена конфигурацией.
2. Задайте длинный пароль или включите двухфакторную аутентификацию.
3. Включите registry платформы и убедитесь, что он требует аутентификацию. Образы публичной организации (в GitLab — группы) читаются без входа, поэтому организация для работ создаётся приватной.
4. Добавьте открытый ключ и проверьте SSH-доступ: `ssh -T -p 2222 git@node-1` (порт 22, если платформа установлена пакетом на хост).

Адрес registry зависит от платформы:

Платформа	Адрес registry
Forgejo	`node-1:8443` — совпадает с адресом интерфейса
Gitea	`node-1:8443` — совпадает с адресом интерфейса
GitLab CE	`node-1:5050` — отдельный сервис

Таблица 3.1: Адрес registry по платформам.

Клиент `docker login` и публикация образов относятся к ЛР3. Результат ЛР2 — интерфейс открывается по HTTPS без предупреждений браузера, SSH отвечает.

Модель угроз

Файл `THREATS.md` заводится в ЛР1 в репозитории инфраструктуры и ведётся до конца семестра. Каждая работа закрывает закреплённые за ней строки; при сдаче предъявляется место, где реализована мера.

Формат файла

`THREATS.md` — таблица с семью колонками. Третья колонка таблицы ниже — пример заполнения строки T1 по итогам ЛР1. Под таблицей угроз допускаются разделы прозой: в ЛР2 туда пишется порядок ротации утёкшего секрета.

Колонка	Что в ней	Пример: строка T1
ID	идентификатор строки: `T1`, `T2`,…; собственные угрозы — `S1`, `S2`,…	`T1`
Актив	что защищаем: узлы, репозиторий, образ, конвейер, кластер, данные	узлы стенда
Угроза (STRIDE)	что может случиться и категория: S, T, R, I, D, E	кража SSH-доступа (S)
Вектор	как именно этого добиваются	подбор пароля, украденный ключ
Мера	что закрывает угрозу	вход только по ключу, firewall по требованиям
Где реализовано	файл, шаг работы или настройка; заполняется студентом	файл первичной настройки, правила ufw
Статус	`открыто` или `закрыто`	`закрыто`

Таблица 4.1: Колонки `THREATS.md` и пример заполнения.

Категории STRIDE: **S** — подмена субъекта, **T** — искажение данных или кода, **R** — отрицание действия, **I** — раскрытие информации, **D** — отказ в обслуживании, **E** — повышение привилегий.

Исходная таблица угроз

Таблица угроз курса — 21 строка. Перенесите все три таблицы в `THREATS.md` одной таблицей, добавьте колонки «Где реализовано» и «Статус» (начальное значение — `открыто`) и закрывайте строки по мере сдачи работ.

Стенд, репозиторий и образ.

ID	Актив	Угроза (STRIDE)	Вектор	Мера	Закрывает
T1	узлы стенда	кража SSH-доступа (S)	подбор пароля, украденный ключ	вход только по ключу, firewall по требованиям	ЛР1
T2	репозиторий	секрет в репозитории (I)	коммит с ключом, секрет в истории	проверка секретов до коммита, ротация утёкшего	ЛР2
T3	репозиторий	подмена кода в незащищённой ветке (T)	прямой push в основную ветку	защищённая ветка, изменение через запрос на слияние с ревью	ЛР2
T4	образ приложения	уязвимый базовый образ или зависимость (E)	старый базовый образ, транзитивная зависимость	сканирование, SBOM, закрепление базового образа по digest	ЛР3
T5a	образ приложения	процесс от root в контейнере (E)	образ без пользователя	non-root пользователь в образе	ЛР3
T5b	кластер	привилегии контейнера в кластере (E)	под без ограничений безопасности	securityContext без root и со сброшенными capabilities	ЛР6

Таблица 4.2: Угрозы стенда, репозитория и образа.

Конвейер и инфраструктура как код.

ID	Актив	Угроза (STRIDE)	Вектор	Мера	Закрывает
T6	конвейер	утечка секрета через логи и артефакты (I)	печать переменной, секрет в артефакте сборки	секреты в переменных CI, маскирование вывода	ЛР4
T7	конвейер	компрометация зависимости конвейера (T)	внешний шаг по плавающему тегу	внешние шаги закреплены по SHA	ЛР4
T8a	образ в registry	подмена артефакта между сборкой и выкаткой (T)	запись в registry, переезд тега	подпись образа в конвейере	ЛР4
T8b	кластер	приём неподписанного образа (T)	развёртывание чужого образа	политика допуска проверяет подпись	ЛР7, дополнительное
T9	код инфраструктуры	секреты в IaC и state (I)	пароль в playbook, state в Git	шифрование секретов в репозитории, state вне Git	ЛР5

T10a	узлы и платформа	дрейф узлов и платформы (T)	ручная правка на узле или в интерфейсе	идемпотентный код, plan показывает дрейф	ЛР5
T10b	кластер	дрейф кластера, ручные правки (T)	правка объекта в обход Git	кластер приводится к состоянию Git	ЛР7

Таблица 4.3: Угрозы конвейера и инфраструктуры как кода.

Кластер, доставка и наблюдаемость.

ID	Актив	Угроза (STRIDE)	Вектор	Мера	Закрывает
T11a	кластер	избыточные права ServiceAccount приложения (E)	учётная запись по умолчанию с широкими правами	минимальные права RBAC	ЛР6
T11b	кластер	избыточный доступ доставки к кластеру (E)	административный kubeconfig в CI	pull без kubeconfig в CI; при push — токен с правами только на namespace	ЛР7
T12	кластер	латеральное движение между подами (E)	скомпрометированный под ходит к БД и кешу	NetworkPolicy по умолчанию запрещает трафик	ЛР6, дополнительное
T13	репозиторий окружений	секрет открытым текстом (I)	Secret закоммичен как есть	секреты запечатаны	ЛР7
T14	данные и доступность	отказ узла, потеря данных (D)	выключение узла с БД	реплики по узлам, PDB, оператор БД с двумя экземплярами	ЛР8
T15	логи и дашборды	персональные данные и секреты в логах, открытая Grafana (I)	логирование тела запроса, интерфейс без аутентификации	политика полей логов, аутентификация Grafana	ЛР8
T16	сервис	незамеченный инцидент (D)	отказ без алерта и без разбора	алерт, runbook, постмортем	ЛР8
T17	репозиторий окружений	токен CI с правом записи (E)	токен с широкой областью действия	доступ конвейера ограничен репозиторием, ветка защищена	ЛР7

Таблица 4.4: Угрозы кластера, доставки и наблюдаемости.

Правила ведения

- В ЛР1 добавьте не менее двух собственных строк, относящихся к варианту и стенду: площадка виртуальных машин, особенности платформы варианта или инструмента доставки. Нумерация — `S1`, `S2`,…; ведутся наравне со строками таблицы курса.
- К сдаче работы N строки основного задания, закреплённые за этой работой, должны иметь статус `закрыто`, а колонка «Где реализовано» — ссылаться на файл или шаг, который открывается при сдаче.
- Строки T8b и T12 закрываются при сдаче дополнительного задания. Открытая строка дополнительного задания баллы основного не снижает.
- Формулировка меры содержит файл, правило или настройку. Отметка «настроено» мерой не считается.

Инструкция агенту

`AGENTS.md` — инструкция ИИ-агенту, лежащая в корне репозитория инфраструктуры. Агент читает её перед работой; в файле описаны параметры стенда, команды проверки результата и запреты. Файл заводится в ЛР1 и дополняется в каждой работе: новый инструмент добавляет строку инвариантов и команду проверки.

Образец

Файл состоит из трёх разделов. Шапка и инварианты:

```
# AGENTS.md — стенд курса «Технологии DevOps», вариант N

## Инварианты стенда

- Два узла Debian 13: `node-1` (платформа, registry, runner, control plane) и
  `node-2` (worker). Обращаться к ним только по именам, адреса не хардкодить.
- Вход по SSH только по ключу; `sudo` спрашивает пароль. Пароли и ключи в
  репозитории не хранятся.
- Порты снаружи: 22, 80, 443 на обоих узлах; 2222, 8443, 5050, 6443 — на
  `node-1`. Новый порт наружу не открывать.
- Всё, что меняется на узлах, описывается кодом в этом репозитории; ручная
  правка на узле — дрейф, её нужно перенести в код.
- Инструменты варианта фиксированы на семестр: менять их нельзя.
```

Команды проверки результата; список дополняется в каждой работе:

```
## Команды проверки

- `ssh student@node-1 'docker compose ps'` — стек приложения жив.
- `ansible-playbook site.yml --check` — код и узлы совпадают.
- `<линтер приложения>` и `<тесты приложения>` — из README приложения.
- `curl -fsS https://node-1:8443/` — платформа отвечает (свой эндпоинт
  проверки: `/api/healthz` у Forgejo и Gitea, `/-/health` у GitLab).
- Изменение считается готовым, только когда команда проверки отработала.
```

Запреты:

```
## Запреты

- Не передавать агенту секреты: приватные ключи, пароли, токены CI, содержимое
  `ansible-vault` и `.env`.
- Не выполнять на узлах команды, которых нет в дельте: сначала показать
  правки, потом применять.
- Не отключать проверки: TLS, подпись образов, гейты конвейера, защиту ветки.
- Не работать агентом в namespace `prod` и в репозитории окружений без ревью.
- Не коммитить сгенерированный код без чтения: ответственность за содержимое
  репозитория несёт студент.
```

Правила ведения

- Файл обновляется вместе со стендом: после ЛР6 в инвариантах появляются дистрибутив и namespace, после ЛР4 — команда просмотра последнего прогона конвейера.
- Раздел «Команды проверки» важнее раздела с описанием: ошибку агента ловит выполненная команда, а не подробная инструкция.
- Секреты в контекст агента не передаются. Ключ, показанный агенту, считается утёкшим и подлежит ротации; порядок ротации проверяется при сдаче ЛР2.
- Изменения, предложенные агентом, проверяются построчно. При сдаче студент отвечает за каждую строку независимо от того, кем она написана.